10. Keylime 웹 API 및 통신 프로토콜 분석

10.1 개요

Keylime은 TPM 기반 원격 증명을 위한 분산 시스템으로, 각 컴포넌트 간의 안전한 통신을 위해 TLS 기반의 RESTful API와 상호 TLS(mTLS) 인증을 사용합니다. 이 문서는 Keylime의 웹 API 구조, TLS 설정, 인증서 관리, 그리고 통신 프로토콜을 상세히 분석합니다.

10.2 TLS 설정 및 인증서 관리

10.2.1 TLS 디렉토리 관리 (`keylime/web_util.py`)

`get_tls_dir()` 함수 동작

```
def  get_tls_dir(component: str) -> str:

tls_dir = config.get(component, "tls_dir")

if tls_dir == "generate":

if component == "verifier":

generatedir = "cv_ca"

elif component == "registrar":

generatedir = "reg_ca"

tls_dir = os.path.abspath(os.path.join(config.WORK_DIR, generatedir))

elif tls_dir == "default":

# 모든 컴포넌트가 verifier의 인증서를 공유
```

```
tls_dir = os.path.abspath(os.path.join(config.WORK_DIR, "cv_ca"))
```

주요 기능:

- **자동 생성 모드**: `tls_dir="generate"` 로 설정 시 컴포넌트별 CA 디렉토리 생성
- **공유 모드**: `tls_dir="default"` 로 설정 시 verifier CA를 모든 컴포넌트가 공유
- **사용자 정의**: 상대 경로 지정 시 `WORK_DIR` 기준으로 절대 경로 변환

`init_tls_dir()` 함수 동작

```
def  init_tls_dir(component: str, logger: Optional[Logger] = None) -> str:
if tls_dir == "generate":
# CA 인증서 경로 확인
ca_path = os.path.join(tls_dir, "cacert.crt")
if os.path.exists(ca_path):
logger.info("기존 CA 인증서 발견: %s", tls_dir)
else:
logger.info("새로운 CA, 키, 인증서 생성: %s", tls_dir)
ca_util.cmd_init(tls_dir) # CA 초기화
# 서버 인증서 생성
ca_util.cmd_mkcert(tls_dir, "server", password=server_key_pw)
# Verifier의 경우 클라이언트 인증서도 생성
if component == "verifier":
ca_util.cmd_mkcert(tls_dir, "client", password=client_key_pw)
```

인증서 생성 과정:

1. **CA 초기화**: `ca_util.cmd_init()` 로 루트 CA 생성
2. **서버 인증서**: 각 컴포넌트용 서버 인증서 및 개인키 생성
3. **클라이언트 인증서**: Verifier용 클라이언트 인증서 생성 (mTLS용)

10.2.2 TLS 컨텍스트 생성

`generate_tls_context()` 함수

```
def generate_tls_context(
certificate: Optional[str],
private_key: Optional[str],
trusted_ca: List[str],
private_key_password: Optional[str] = None,
verify_peer_cert: bool = True,
is_client: bool = False,
ca_cert_string: Optional[str] = None,
logger: Optional[Logger] = None,
) -> ssl.SSLContext:
```

TLS 설정 특징:

- **TLS 1.2 최소 버전**: `context.minimum_version = ssl.TLSVersion.TLSv1_2`
- **호스트명 검증 비활성화**: `context.check_hostname = False` (IP 기반 통신)
- **상호 인증**: `context.verify_mode = ssl.CERT_REQUIRED` (mTLS 활성화 시)
- **임시 인증서 지원**: Agent의 자체 서명 인증서를 임시 파일로 처리

10.3 상호 TLS(mTLS) 인증 구현

10.3.1 mTLS 초기화 (`init_mtls()`)

```
def  init_mtls(component: str, logger: Optional[Logger] = None) -> ssl.SSLContex
# TLS 디렉토리 초기화
_ = init_tls_dir(component, logger=logger)
# TLS 옵션 획득
(cert, key, trusted_ca, pw), verify_client = get_tls_options(component, logger=l
# 서버 TLS 컨텍스트 생성
return generate_tls_context(
cert, key, trusted_ca,
private_key_password=pw,
verify_peer_cert=verify_client,
logger=logger
)
```

10.3.2 Agent mTLS 컨텍스트 (`generate_agent_tls_context()`)

```
def  generate_agent_tls_context(
component: str, cert_blob: str, logger: Optional[Logger] = None
) -> Optional[ssl.SSLContext]:
# Agent mTLS 활성화 확인
```

```
agent_mtls_enabled = config.getboolean(component, "enable_agent_mtls")

if not agent_mtls_enabled:

return None

# Agent 자체 서명 인증서를 임시 파일로 저장

with tempfile.TemporaryDirectory(prefix="keylime_") as tmp_dir:

agent_cert_file = os.path.abspath(os.path.join(tmp_dir, "agent.crt"))

with  open(agent_cert_file, "wb") as f:

f.write(cert_blob.encode())

# 신뢰할 수 있는 CA 목록에 Agent 인증서 추가

trusted_ca.append(agent_cert_file)
```

mTLS 보안 특징:

- **양방향 인증**: 클라이언트와 서버 모두 인증서로 신원 확인
- **동적 인증서**: Agent의 자체 서명 인증서를 런타임에 신뢰 목록에 추가
- **임시 파일 처리**: 보안을 위해 임시 디렉토리에서 인증서 처리

10.4 RESTful API 엔드포인트 및 메시지 형식

10.4.1 API 요청 파라미터 파싱 (`get_restful_params()`)

```
def  get_restful_params(urlstring: str) -> Dict[str, Union[str, None]]:

parsed_path = urllib.parse.urlsplit(urlstring.strip("/"))

query_params = urllib.parse.parse_qsl(parsed_path.query)

path_tokens = parsed_path.path.split("/")
```

```
# API 버전 검증 및 추출
api_version = "0"
if path_tokens[0] and re.match(r"^v?[0-9]+(\.[0-9]+)?", path_tokens[0]):
version = keylime_api_version.normalize_version(path_tokens[0])
if keylime_api_version.is_supported_version(version):
api_version = version
path_tokens.pop(0)
# 경로를 키-값 쌍으로 변환
path_params = _list_to_dict(path_tokens)
path_params["api_version"] = api_version
path_params.update(query_params)
return path_params
```

10.4.2 표준 JSON 응답 형식 (echo_json_response())

```
def echo_json_response(
handler: Any, code: int, status: Optional[str] = None,
results: Optional[Dict[str, Any]] = None
) -> bool:
json_res = {"code": code, "status": status, "results": results}
json_response = json.dumps(json_res)
# Tornado 또는 BaseHTTPRequestHandler 지원
if isinstance(handler, tornado.web.RequestHandler):
```

```
handler.set_status(code)

handler.set_header("Content-Type", "application/json")

handler.write(json_response_bytes)

handler.finish()
```

표준 응답 구조:

```
{

"code": 200,

"status": "Success",

"results": {

"agent_id": "example-uuid",

"operational_state": 7,

"data": "..."

}

}
```

10.4.3 주요 API 엔드포인트 패턴

Registrar API (`keylime/registrar_client.py`)

```
# Agent 데이터 조회

GET /v{API_VERSION}/agents/{agent_id}


# 응답 데이터 구조 (RegistrarData)
```

```
{
"ip": "192.168.1.100",
"port": "9002",
"regcount": 1,
"mtls_cert": "-----BEGIN CERTIFICATE-----...",
"aik_tpm": "base64_encoded_aik",
"ek_tpm": "base64_encoded_ek",
"ekcert": "base64_encoded_ekcert",
"provider_keys": {"tpm_manufacturer": "key_data"}
}
```

Verifier API 패턴

```
# Agent 등록/관리
POST /v{API_VERSION}/agents/{agent_id}
GET /v{API_VERSION}/agents/{agent_id}
PUT /v{API_VERSION}/agents/{agent_id}
DELETE /v{API_VERSION}/agents/{agent_id}

# 증명 상태 조회
GET /v{API_VERSION}/agents/{agent_id}/status
```

10.5 API 버전 관리 (`keylime/api_version.py`)

10.5.1 버전 관리 시스템

```
CURRENT_VERSION: str = "2.4"

VERSIONS: List[str] = ["1.0", "2.0", "2.1", "2.2", "2.3", "2.4"]

LATEST_VERSIONS: Dict[str, str] = {"1": "1.0", "2": "2.4"}

DEPRECATED_VERSIONS: List[str] = ["1.0"]
```

10.5.2 버전 검증 및 정규화

```
def normalize_version(version_type: VersionType) -> str:

version_type = str(version_type).strip("/")

try:

base_version = version.parse(version_type).base_version

except version.InvalidVersion:

return version_type

# 단일 숫자인 경우 최신 마이너 버전 반환

if "." not in base_version:

latest_minor = latest_minor_version(base_version)

if latest_minor != "0":

return latest_minor

return base_version


def validate_api_version(handler: Any, version: VersionType, logger: Logger) ->
```

```
if not version or not keylime_api_version.is_supported_version(version):
echo_json_response(handler, 400, "API Version not supported")
return False
if keylime_api_version.is_deprecated_version(version):
logger.warning("클라이언트 요청 API 버전 %s는 deprecated됨", version)
return True
```

버전 관리 특징:

- **하위 호환성**: 이전 버전 API 지원
- **자동 업그레이드**: 메이저 버전만 지정 시 최신 마이너 버전 사용
- **Deprecation 경고**: 구버전 사용 시 로그 경고

10.6 HTTP 클라이언트 구현

10.6.1 RequestsClient (`keylime/requests_client.py`)

```
class RequestsClient:
def __init__(
self,
base_url: str,
tls_enabled: bool,
tls_context: Optional[ssl.SSLContext] = None,
ignore_hostname: bool = True,
**kwargs: Any,
```

```
) -> None:

# URL 정규화

if base_url.startswith("http"):

base_url = re.sub(r"https?://", "", base_url)

self.base_url = f"https://{base_url}"  if tls_enabled else  f"http://{base_url}"

# TLS 어댑터 설정

if tls_enabled:

self.session.mount("https://", HostNameIgnoreAdapter(tls_context, ignore_hostnam
```

10.6.2 Tornado 비동기 클라이언트 (`keylime/tornado_requests.py`)

```
async  def  request(

method: str, url: str,

params: Optional[Dict[str, str]] = None,

data: Optional[Dict[str, Any]] = None,

context: Optional[ssl.SSLContext] = None,

headers: Optional[Union[Dict[str, str], HTTPHeaders]] = None,

timeout: float = 60.0,

) -> TornadoResponse:

# JSON 데이터 변환

if  isinstance(data, dict):

body = json.dumps(data)

if headers is None:
```

```
headers = {}

if  "Content-Type" not in headers:

headers["Content-Type"] = "application/json"

# 비동기 HTTP 요청

req = httpclient.HTTPRequest(

url=url, method=method, ssl_options=context,

body=body, headers=headers, request_timeout=timeout,

)

response = await http_client.fetch(req)
```

10.7 보안 및 오류 처리

10.7.1 TLS 보안 설정

```
# TLS 1.2 최소 버전 강제

if sys.version_info >= (3, 7):

context.minimum_version = ssl.TLSVersion.TLSv1_2

else:

context.options &= ~ssl.OP_NO_TLSv1_2


# 피어 인증서 검증 활성화

context.verify_mode = ssl.CERT_REQUIRED

context.check_hostname = False # IP 기반 통신을 위해 비활성화
```

10.7.2 인증서 강도 검증

```
except ssl.SSLError as exc:
if exc.reason == "EE_KEY_TOO_SMALL" and logger:
logger.error(
"이 시스템에서는 더 높은 키 강도가 필요합니다. "
"keylime이 인증서를 생성하는 경우 [ca]cert_bits 설정값을 "
"높이고 생성된 인증서를 제거한 후 keylime 서비스를 재실행하세요"
)
raise exc
```

10.7.3 Base64 URL-Safe 디코딩

```
def  urlsafe_nopad_b64decode(encoded_bytes: bytes) -> bytes:
# 패딩이 없는 경우 필요한 패딩 추가
len_pad = len(encoded_bytes) % 4
if len_pad != 0:
encoded_bytes += b"=" * (4 - len_pad)
return base64.urlsafe_b64decode(encoded_bytes)
```

10.8 통신 프로토콜 흐름

10.8.1 Agent 등록 프로토콜

```
1. Agent → Registrar: POST /v2.4/agents/{agent_id}

- EK, AIK, mTLS 인증서 전송

2. Registrar → Agent: 200 OK

- 등록 확인 및 regcount 반환

3. Tenant → Registrar: GET /v2.4/agents/{agent_id}

- Agent 공개키 정보 조회

4. Tenant → Verifier: POST /v2.4/agents/{agent_id}

- Agent 검증 등록 요청
```

10.8.2 지속적 증명 프로토콜

```
1. Verifier → Agent: GET /v2.4/quotes/integrity (mTLS)

- PCR 값 및 IMA 측정값 요청

2. Agent → Verifier: 200 OK

- TPM Quote, PCR 값, IMA 로그 응답

3. Verifier: 내부 검증 프로세스

- TPM Quote 서명 검증

- PCR 값 정책 비교

- IMA 측정값 검증

4. Verifier → Tenant: Notification (실패 시)

- 무결성 위반 알림
```

10.9 성능 및 확장성 고려사항

10.9.1 연결 풀링

```
class HostNameIgnoreAdapter(HTTPAdapter):

def  init_poolmanager(self, connections: int, maxsize: int, **pool_kwargs: Any)

self.poolmanager = PoolManager(

num_pools=connections,

maxsize=maxsize,

block=block,

strict=True,

assert_hostname=assert_hostname,

ssl_context=self._tls_context,

**pool_kwargs,

)
```

10.9.2 비동기 처리

- **Tornado 프레임워크**: 비동기 I/O로 높은 동시성 지원
- **ThreadPoolExecutor**: CPU 집약적 작업의 스레드 풀 처리
- **Connection Reuse**: HTTP 연결 재사용으로 성능 최적화

10.10 설정 및 배포 고려사항

10.10.1 TLS 설정 옵션

```
[verifier]
tls_dir = generate # 자동 생성 | default | 사용자 경로
server_cert = default
server_key = default
trusted_client_ca = default
enable_agent_mtls = True

[tenant]
client_cert = default
client_key = default
trusted_server_ca = default
```

10.10.2 네트워크 설정

```
[verifier]
ip = 127.0.0.1
port = 8881
request_timeout = 60

[registrar]
ip = 127.0.0.1
port = 8890
```

이 분석을 통해 Keylime의 웹 API와 통신 프로토콜이 어떻게 TPM 기반 원격 증명을 위한 안전하고 확장 가능한 분산 시스템을 구현하는지 이해할 수 있습니다. mTLS 기반의 강력한 보안과 RESTful API의 단순성을 결합하여 엔터프라이즈급 보안 솔루션을 제공합니다.